

# Algorithmen 3

## Übungen

# HINWEIS ZU DEN UNIT TESTS

Für alle deine Unit Tests gilt:

- 100 % Testabdeckung
- AAA-Pattern
- Mindestens einen zusätzlichen Edge Case  
(d. h. du hast dann theoretisch mehr als 100 % Testabdeckung)

# GroupWordsByLength

Gegeben ist eine Liste von Wörtern. Schreibe eine Methode, die die Wörter nach ihrer Länge gruppiert und eine Map zurückgibt, wobei die Schlüssel die Länge der Wörter sind und die Werte eine Liste von Wörtern dieser Länge sind.

**Beispiel:**

```
public static void main(String[] args) {  
    List<String> words = List.of("apple", "banana", "orange", "pear", "grape", "kiwi");  
  
    Map<Integer, List<String>> groupedWords = groupWordsByLength(words);  
  
    groupedWords.forEach((length, wordList) -> System.out.println("Words of length " + length + ": " + wordList));  
}
```

**Ausgabe:**

```
Words of length 4: [pear, kiwi]  
Words of length 5: [apple, grape]  
Words of length 6: [banana, orange]
```

# mergeSortedLists

Implementiere eine Methode, die **zwei SORTIERTE Listen** von ganzen Zahlen **fusioniert**, um eine **neue sortierte Liste** zu erzeugen.

Verwende dazu List und implementiere den Algorithmus **ohne** die Verwendung von zusätzlichen Datenstrukturen (z. B. TreeSet) oder Sortieralgorithmen (z. B. Arrays.sort, Collections.sort, Stream.sorted, BubbleSort, MergeSort, QuickSort etc).

**Beispiel:**

```
public static void main(String[] args) {  
    List<Integer> sortedList1 = List.of(1, 2, 5, 7, 9); // sortiert!  
    List<Integer> sortedList2 = List.of(3, 4, 6, 8, 9, 10); // sortiert!  
  
    List<Integer> mergedList = mergeSortedLists(sortedList1, sortedList2);  
  
    System.out.println("Merged List: " + mergedList);  
}
```

**Ausgabe:**

```
Merged List: [1, 2, 3, 4, 5, 6, 7, 8, 9, 9, 10]
```

# Anagramm-Gruppen

Schreibe eine Methode, die eine Liste von Wörtern akzeptiert und Gruppen von Anagrammen identifiziert. Zwei Wörter gelten als Anagramme, wenn sie dieselben Buchstaben in unterschiedlicher Reihenfolge enthalten.

**Beispiel:**

```
public static void main(String[] args) {  
    List<String> wordList = List.of("listen", "silent", "enlist", "eat", "tea", "ate", "hello", "world");  
  
    List<List<String>> anagramGroups = identifyAnagramGroups(wordList);  
  
    anagramGroups.forEach(group -> System.out.println("Anagrammgruppe: " + group));  
}
```

**Ausgabe:**

```
Anagrammgruppe: [eat, tea, ate]  
Anagrammgruppe: [listen, silent, enlist]  
Anagrammgruppe: [hello]  
Anagrammgruppe: [world]
```

# User Authority

Schreibe eine Klasse **User**, die als Attribut authority (String) besitzt.

Füge dem User folgende Methoden hinzu:

```
public boolean hasAuthority(String authority) // prüft, ob der User die Authority hat
```

```
public boolean hasRole(String role) // prüft, ob der User die Rolle hat
```

Beide Methoden vergleichen das Attribut authority mit dem übergebenen Parameter. Die Methode hasRole prüft aber, ob das Prefix "ROLE\_" am Anfang des Parameters existiert. Falls nicht, wird dies am Anfang des übergebenen Parameters **vor dem Vergleich** hinzugefügt.

Beispiel:

```
new User("ROLE_ADMIN").hasAuthority("ROLE_ADMIN"); // true
```

```
new User("ROLE_ADMIN").hasRole("ADMIN"); // true
```

Nutze den nachfolgenden **Unit Test**, um deine Lösung zu testen.

# User Authority: Unit Test

```
class UserTest {

    @Test
    void hasAuthorityWithMatchingAuthority() {
        // Arrange
        User user = new User("ROLE_ADMIN");

        // Act
        boolean result = user.hasAuthority("ROLE_ADMIN");

        // Assert
        assertTrue(result);
    }

    @Test
    void hasAuthorityWithNonMatchingAuthority() {
        // Arrange
        User user = new User("ROLE_ADMIN");

        // Act
        boolean result = user.hasAuthority("ADMIN");

        // Assert
        assertFalse(result);
    }

    @Test
    void hasAuthorityEmpty() {
        // Arrange
        var user = new User("ROLE_ADMIN");

        // Act
        var actual = user.hasAuthority("");

        // Assert
        assertFalse(actual);
    }

    @Test
    void hasAuthorityIsCaseSensitive() {
        // Arrange
        var user = new User("ROLE_ADMIN");

        // Act
        var actual = user.hasAuthority("roLe_AdMiN");

        // Assert
        assertFalse(actual);
    }
}
```

```
@Test
void hasRoleWithMatchingRole() {
    // Arrange
    User user = new User("ROLE_ADMIN");
    // Act
    boolean result = user.hasRole("ADMIN");
    // Assert
    assertTrue(result);
}

@Test
void hasRoleWithNonMatchingRole() {
    // Arrange
    User user = new User("ROLE_ADMIN");
    // Act
    boolean result = user.hasRole("USER");
    // Assert
    assertFalse(result);
}

@Test
void hasRoleWithPrefixIncluded() {
    // Arrange
    User user = new User("ROLE_ADMIN");
    // Act
    boolean result = user.hasRole("ROLE_ADMIN");
    // Assert
    assertTrue(result);
}

@Test
void hasRoleEmpty() {
    // Arrange
    var user = new User("ROLE_ADMIN");
    // Act
    var actual = user.hasRole("");
    // Assert
    assertFalse(actual);
}

@Test
void hasRoleIsCaseSensitive() {
    // Arrange
    var user = new User("ROLE_ADMIN");
    // Act
    var actual = user.hasRole("AdMiN");
    // Assert
    assertFalse(actual);
}
}
```

# PasswordManager

Schreibe eine Klasse **PasswordManager**. Füge ein Konstruktor und folgende Methode hinzu:

```
public PasswordManager(String password) { ... } // store password in attribute
```

```
public boolean matchesStoredPassword(String password) { ... } // compare password with this.password
```

Der Konstruktor und `matchesStoredPassword` prüfen zunächst, ob das übergebene Password **kürzer** als **64 Zeichen** ist. Falls ja, wird es so lange mit **"#"-Zeichen** möglichst **effizient** aufgefüllt (Erinnerung: String-Konkatenation erstes Semester), bis es die maximale Länge von 64 Zeichen hat.

Falls es länger als 64 Zeichen ist, wird das Passwort auf 64 Zeichen gekürzt.

Der Konstruktor speichert das modifizierte Passwort in einem Attribut.

Die Methode `matchesStoredPassword` vergleicht ihr modifiziertes Passwort mit dem gespeicherten Passwort (im Attribut) und gibt das Ergebnis zurück (true oder false).

**Überlege** dir, warum der PasswordManager das Auffüllen macht.

Nutze den nachfolgenden **Unit Test** um deine Lösung zu testen.



# PasswordManager: Unit Test

```
class PasswordManagerTest {
```

```
    @Test
    void storedPasswordLongerThan64CharactersIsTruncated() {
        // Arrange
        var password1 = "a".repeat(65);
        var password2 = "a".repeat(64);

        // Act
        var actual = new PasswordManager(password1).matchesStoredPassword(password2);

        // Assert
        assertTrue(actual);
    }
```

```
    @Test
    void inputPasswordLongerThan64CharactersIsTruncated() {
        // Arrange
        var password1 = "a".repeat(64);
        var password2 = "a".repeat(65);

        // Act
        var actual = new PasswordManager(password1).matchesStoredPassword(password2);

        // Assert
        assertTrue(actual);
    }
```

```
    @Test
    void samePasswordsAreEqual() {
        // Arrange
        var password1 = "password";
        var password2 = "password";

        // Act
        var actual = new PasswordManager(password1).matchesStoredPassword(password2);

        // Assert
        assertTrue(actual);
    }
```

```
    @Test
    void differentPasswordsSameLengthAreNotEqual() {
        // Arrange
        var password1 = "password1";
        var password2 = "password2";

        // Act
        var actual = new PasswordManager(password1).matchesStoredPassword(password2);

        // Assert
        assertFalse(actual);
    }
```

```
    @Test
    void differentPasswordsDifferentLengthAreNotEqual() {
        // Arrange
        var password1 = "password";
        var password2 = "pass";

        // Act
        var actual = new PasswordManager(password1).matchesStoredPassword(password2);

        // Assert
        assertFalse(actual);
    }
```

```
    @Test
    void passwordsAreCaseSensitive() {
        // Arrange
        var password1 = "PaSswOrd";
        var password2 = "pAsSWoRd";

        // Act
        var actual = new PasswordManager(password1).matchesStoredPassword(password2);

        // Assert
        assertFalse(actual);
    }
```

```
    @Test
    void nullStoredPasswordThrowsException() {
        assertThrows(IllegalArgumentException.class, () -> new PasswordManager(null).matchesStoredPassword("password"));
    }
```

```
    @Test
    void nullInputPasswordThrowsException() {
        assertThrows(IllegalArgumentException.class, () -> new PasswordManager("password").matchesStoredPassword(null));
    }
```

```
    @Test
    void emptyPasswordAreEqual() {
        // Arrange
        String password1 = "";
        String password2 = "";

        // Act
        var actual = new PasswordManager(password1).matchesStoredPassword(password2);

        // Assert
        assertTrue(actual);
    }
}
```